

Overflow

The flags: CF, OF
 ↓
 unsigned signed

overflow situations:

$$\begin{array}{r} 0 \dots + \\ 0 \dots \\ \hline 1 \dots \end{array}$$

"+" and "+"
cannot be "-"

$$\begin{array}{r} 1 \dots + \\ 1 \dots \\ \hline 0 \dots \end{array}$$

"-" and "-"
cannot be "+"

$$\begin{array}{r} 1 \dots - \\ 0 \dots \\ \hline 0 \dots \end{array}$$

$$\begin{array}{r} 0 \dots - \\ 1 \dots \\ \hline 1 \dots \end{array}$$

Example: 1 0110 0010 -
 1100 1000

 1001 1010
 CF = 1.
 (imprudent)
 nonsensical result.

B2

$$\begin{array}{r} 98 - \\ 200 \quad (10) \\ \hline -102 \end{array}$$

unsigned, CF = 1.

$$\begin{array}{r} 98 - \\ -56 \\ \hline 154 \end{array}$$

signed, OF = 1

B10

$$\begin{array}{r} 62h - \\ c8h \\ \hline 9Ah \end{array}$$

B16

n mathematically correct,
 but the interpretation is incorrect
 $\Rightarrow$ CF, OF = 1.

*, / ?

① There is no overflow when we multiply.

$$\begin{array}{r} \boxed{n \text{ bits}} * \\ \boxed{n \text{ bits}} \\ \hline \boxed{2n \text{ bits}} \end{array}$$

2.3

Situation when CF = 0 when multiplying:

$$\begin{array}{l} \text{byte} * \text{byte} = b \\ w * w = w \\ d * d = d \\ CF = OF = 0. \end{array}$$

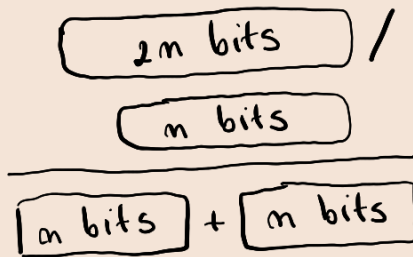
$$\begin{array}{l} b * b = w \\ w * w = d \\ d * d = q \\ CF = OF = 1 \end{array}$$

I need 2n bits to store
 the result (look for the
 results in two registers)

DX: AX $\rightarrow$ a pair of registers $\rightarrow$ CF = OF = 1.

~~mov DX, AX, 0Fh~~

② There can be an overflow when we divide (ex: division by 0)



1000 / 1 :
mov AX, 1000
mov BL, 1
div BL $\rightarrow$ BX

$\rightarrow$ we've triggered an overflow.
"Divide overflow"
"Division by 0"
"Zero divide"

WHY $\rightarrow$ 1000 cannot fit into a byte

1000 / 1
1000 / 0

CF, OF

$\downarrow$
DX: AX ...

IADD, ISUB

~~X~~

EU: execution unit

BIU: fetches from memory, passes to the left

code $\rightarrow$ machine code $\rightarrow$ bring from memory

data $\rightarrow$ DS (data segment)

code $\rightarrow$ CS (code segment)

Registers: closest to CPU. Closer than cache?

EU of CPU:

$\rightarrow$ registers: EAX $\rightarrow$ accumulator reg.

EBX $\rightarrow$ base reg.

ECX $\rightarrow$ counter reg. (for loops, counting, the thing that counts is stored here)

EDX $\rightarrow$ data reg.

ESP $\rightarrow$ stack pointer } STACK reg.

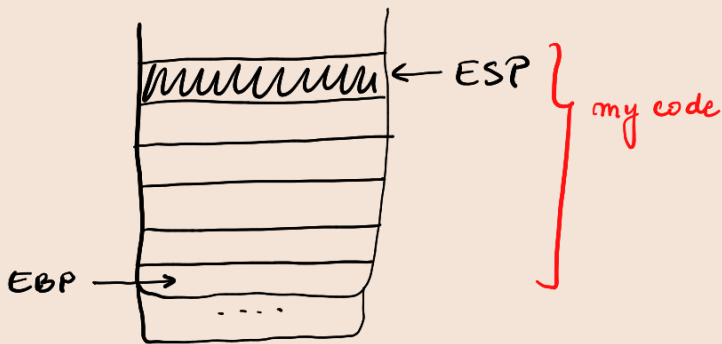
EBP $\rightarrow$ base pointer }

EDI (dest) } INDEX reg.

ESI (source) }

32-bit CPU $\Rightarrow$ we can store dw in a stack.

ESP + 1 } for accessing other things in the stack
EBP + 1 }

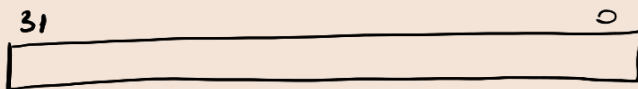


PUSH op. mov EAX, [ESP+8] } later on.
POP op

- EAX** - doubleword address given by EAX
- the lower word: highbyte, lowbyte (AH, AL)
(16 bits) (8 bits) (8 bits)
 - the upper word: ?
 - we can access $\begin{cases} \text{bytes} \\ \text{words} \\ \text{doublewords} \end{cases}$

64-bit CPU : RAX
RBX
...

Flags



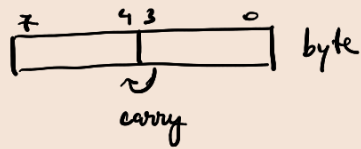
CF: carry flag

PF: parity flag

lower byte of LPO

nr. bits of LPO + PF = odd nr. of bits of 1

AF: auxiliary flag



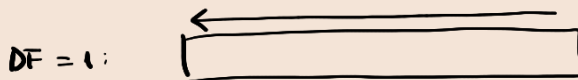
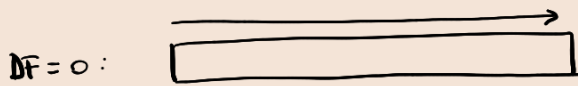
ZF: zero flag, $LPO = 0 \Rightarrow ZF = 1$.

SF: sign flag, $LPO < 0 \Rightarrow SF = 1$.

TF: trap flag, $TF = 1 \Rightarrow$ CPU stops after LPO.

IF: interrupt flag

DF: direction flag



OF: overflow flag (signed)

Categories (flags):

a) flag set as a direct effect of LPO:

CF, AF, PF, ZF, SF, OF

b) flag set by the programmes to influence the way the next instructions are executed

CF, TF, DF, IF.

Instructions to set the flag value:

CF:

STC (set carry), $CF \leftarrow 1$

CLC (clear carry), $CF \leftarrow 0$

DF:

STD, $DF \leftarrow 1$

CID, $DF \leftarrow 0$

IF: Only on 16-bit CPUs:

STI

CLI

CF / Carry Flag

ADD

add regd, regs
add reg, mem
add mem, reg
add reg/mem, ct

operands ← registers
memory
ct

$\begin{array}{r} 11 \\ 1001 \ 0011 (2) + \\ 0111 \ 0011 (2) \end{array}$

$\begin{array}{r} 1 \ 0000 \ 0110 \\ \hline \end{array} \rightarrow \begin{array}{r} 0000 \ 0110 \\ \hline 0 \quad 6 \end{array} = 06h$
CF = 1, unsigned

mov CL, 1001 0011b
add CL, 0111 0011b ; CL = 06h, CF = 1.
mov CL, 93h
mov CL, 0FFh ; CL = -1
↓
0xFF

0000 1111 1111

PF Parity Flag

SUB

sub regd, regs
sub reg, mem
sub mem, reg
sub reg/mem, ct

mov BL, 1000 1100b
add BL, 0000 0010b ; BL ← 1000 1110b
sub BL, 1000 0000b

"CHECK SUMS".
BL ← 1000 1110b
4 is even → PF = 1.
BL ← 0000 1110b
3 is odd → PF = 0.

... data segment:
m db 10
...

... code segment:
...

sub [m], 2

word?
dword?
byte?
⇒ ERROR (we don't know the size of 2.)

the value of m : $[m]=10$.

sub byte $[m], 2 \Rightarrow$ CORRECT

sub BL, 2 ✓
↓ ↓
byte byte

add 2, BL ✗ we don't know the size of 2.

sub BL, BX ✗ we have $\neq$ sizes

sub BL, CL ✓

PUSH 2 $\rightarrow$ 32 bit CPU

↓
stack $\rightarrow$ dw.

PUSH [b] $\Rightarrow$ ERROR
PUSH dword [b] ✓ } to be checked in our program because in newer versions it might not work.

AF Auxiliary flag (Bit 3 $\rightarrow$ Bit 4)

mov AL, 93h
add AL, 73h

AL = 1001 0011b

AL = 1001 0011 + 0111 0011

AL = 0000 0110

no carry from bit 4 $\leftarrow$ bit 3 $\Rightarrow$ AF=0.

ZF

inc reg/mem ; $\text{reg/mem} \leftarrow \text{reg/mem} + 1$
dec reg/mem ; $\text{reg/mem} \leftarrow \text{reg/mem} - 1$

mov ECX, 1

dec ECX ; $\text{ECX} = \text{ECX} - 1 = 0$, ZF = 1

mov EAX, 0FFFFFFFFh

inc EAX ; $\text{EAX} = \text{EAX} + 1$, ZF = 1, CF = 1, SF = 1

SF

mov BL, 1

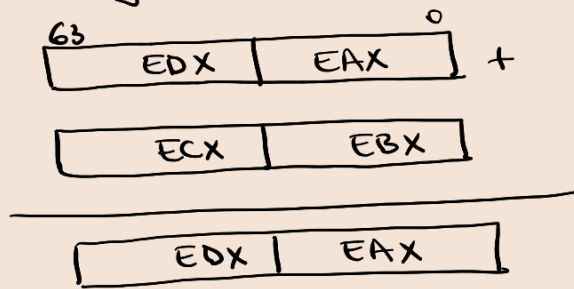
sub BL, 2 ; $\text{BL} \leftarrow \text{FFh}$, SF = 1.

1111 1111

ADC

Adding two quad words stored in EDI:EAX and ECX:EBX

! adding two 32-bit numbers stored in EDI, ESI, and ECX. CBX



add EAX, EBX
adc EDX, ECX

$EDX:EAX \leftarrow EDX:EAX + ECX:EBX$

SBB

movzx reg32, reg8/mem8
reg16, reg8/mem8
reg32, reg16/mem16

→ movzx completes with Zero.

data segment
val db 1000 1111 b ; 8Fh

code segment

movzx AX, [val] ; AX ← 008Fh
movzx EAX, [val] ; EAX ← 0000008Fh

movsx

movsx AX, [val] ; AX ← FF8Fh
movsx EAX, [val] ; EAX ← FFFFFFFF8Fh

movsx completes with SIGN BIT

XCHG

xchg reg, reg
reg, mem
mem, reg

→ changes the label of the memory/register without taking an auxiliary one



mov AL, 8

mov CL, 2

xchg AL, CL ; AL ← 2
CL ← 8